

中山大学计算机院本科生实验报告

(2025 学年春季学期)

课程名称：并程序序设计

批改人：

实验	MPI 点对点通信实现并行矩阵	专业（方向）	计算机科学与技术
学号	22320021	姓名	陈安康
Email	1743826979@qq.com	完成日期	2025/3/26

1. 实验目的

使用 MPI 点对点通信方式实现并行矩阵乘法计算，熟悉 MPI 点对点通信的特点及应用实现方法，分析 MPI 点对点通信对的大规模矩阵计算的性能提升。讨论内存有限情况下的大规模矩阵乘法的可行性方案，并进一步讨论提升大规模稀疏矩阵的计算性能。

2. 实验过程和核心代码

1.MPI 点对点通信实现大规模并行矩阵乘法

MPI（Message Passing Interface）是用于编写并行程序的标准，点对点通信是 MPI 中最基础的通信模式之一，它允许一个进程向另一个进程发送和接收数据。这种通信模式为并行程序中进程之间的数据交换提供了基本手段，是构建更复杂通信模式和并行算法的基础。

点对点通信涉及两个进程之间的直接数据传输，一个进程作为发送方，另一个进程作为接收方。发送方将数据从其本地内存缓冲区发送到网络或共享内存中，接收方则从网络或共享内存中接收数据并存储到其本地内存缓冲区。其 API 是 MPI 提供的 MPI_Send()和 MPI_Recv()函数。

在 Linux 虚拟机上实现核心代码如下，对 A 矩阵进行按行分块，通过点对点通信进行数据分发，并行计算。由于每一个步骤都有强依赖性，所以使用非阻塞点对点通信即可。在计算运行时间时，考虑通信开销，求取所有进程中时间开销最大者作为整个程序的时间开销。

```

int main(int argc, char** argv) {
    MPI_Init(NULL, NULL);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // 输入矩阵规模
    int m = 0, n = 0, k = 0;
    double start_time = MPI_Wtime();
    if (rank == 0) {
        // 确保输入范围在 [128, 2048]
        m = 128, k = 128, n = 128;
        // 发送矩阵规模到其他进程
        for (int i = 1; i < size; ++i) {
            MPI_Send(&m, 1, MPI_INT, i, 0, MPI_COMM_WORLD);
            MPI_Send(&n, 1, MPI_INT, i, 1, MPI_COMM_WORLD);
            MPI_Send(&k, 1, MPI_INT, i, 2, MPI_COMM_WORLD);
        }
    } else {
        // 接收矩阵规模
        MPI_Recv(&m, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Recv(&n, 1, MPI_INT, 0, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Recv(&k, 1, MPI_INT, 0, 2, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("rank %d receive the matrix param\n", rank);
    }
    // 生成矩阵
    double *A = (double *)malloc(m * n * sizeof(double));
    double *B = (double *)malloc(n * k * sizeof(double));
    double *C = (double *)malloc(m * k * sizeof(double));

```

```

if (rank == 0) {
    srand(time(NULL));
    generateRandomMatrix(A, m, n);
    generateRandomMatrix(B, n, k);
    printf("init matrix successfully!!\n");
    // 发送矩阵 B 到其他进程
    for (int i = 1; i < size; ++i) {
        MPI_Send(B, n * k, MPI_DOUBLE, i, 3, MPI_COMM_WORLD);
    }
} else {
    // 接收矩阵 B
    MPI_Recv(B, n * k, MPI_DOUBLE, 0, 3, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("rank %d have receive the matrix B\n",rank);
}

// 分发矩阵 A 的部分到各个进程
int local_m = m / size;
double *local_A = (double *)malloc(local_m * n * sizeof(double));
if (rank == 0) {
    for (int i = 0; i < size; ++i) {
        int start = i * local_m * n;
        if (i == 0) {
            for (int j = 0; j < local_m * n; ++j) {
                local_A[j] = A[j];
            }
        } else {
            MPI_Send(A + start, local_m * n, MPI_DOUBLE, i, 4, MPI_COMM_WORLD);
        }
    }
} else {
    MPI_Recv(local_A, local_m * n, MPI_DOUBLE, 0, 4, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("rank %d have receive the local_A\n",rank);
}

// 计算局部结果
double *local_C = (double *)malloc(local_m * k * sizeof(double));
matrixMultiply(local_A, B, local_C, local_m, n, k);

// 收集局部结果到进程 0
if (rank == 0) {
    for (int i = 0; i < size; ++i) {
        int start = i * local_m * k;
        if (i == 0) {
            for (int j = 0; j < local_m * k; ++j) {
                C[j] = local_C[j];
            }
        } else {
            MPI_Recv(C + start, local_m * k, MPI_DOUBLE, i, 5, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        }
    }
} else {
    MPI_Send(local_C, local_m * k, MPI_DOUBLE, 0, 5, MPI_COMM_WORLD);
}
double end_time = MPI_Wtime();
double local_time = end_time - start_time;
// 收集所有进程的时间
double *all_times = (double *)malloc(size * sizeof(double));
if (rank == 0) {
    all_times[0] = local_time;
    for (int i = 1; i < size; ++i) {
        MPI_Recv(all_times + i, 1, MPI_DOUBLE, i, 6, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }
} else {
    MPI_Send(&local_time, 1, MPI_DOUBLE, 0, 6, MPI_COMM_WORLD);
}

```

编译和运行命令如下，由于我的虚拟机核数过少，仅有两个核，所以需要
使用--oversubscribe 选项。

```

进程数: 1 | 矩阵规模: 2048 | 矩阵计算所消耗的时间: 189.753563 秒
user@user-virtual-machine:~/并行程序设计/hw2$ mpicc -g -Wall -o matrix matrix.c
user@user-virtual-machine:~/并行程序设计/hw2$ mpiexec -n 1 --oversubscribe ./mat
rix

```

由于实验结果过多，这里就不一一截图展示了，在实验中，我发现，在我的电脑上运行时，比较 1 个进程和 2 个进程的运行时间，会发现运行时间有很大提示，可是进程数再增加，运行时间完全不会有任何进步，反而会开销更大。如下图：

```

886 512.392995 527.677817 505.792893 506.088065
进程数: 1 | 矩阵规模: 2048 | 矩阵计算所消耗的时间: 189.753563 秒
user@user-virtual-machine:~/并行程序设计/hw2$

```

2 进程下：

```

710 514.483092 500.744842 505.355517 486.504446
进程数: 2 | 矩阵规模: 2048 | 矩阵计算所消耗的时间: 110.005584 秒
user@user-virtual-machine:~/并行程序设计/hw2$

```

4 进程下：

```

512 501.078895 507.560619 507.204369 499.498334
进程数: 4 | 矩阵规模: 2048 | 矩阵计算所消耗的时间: 112.949566 秒
user@user-virtual-machine:~/并行程序设计/hw2$

```

8 进程下：

```

448 516.206415 518.447090 525.941262 510.373329
进程数: 8 | 矩阵规模: 2048 | 矩阵计算所消耗的时间: 113.155960 秒
user@user-virtual-machine:~/并行程序设计/hw2$

```

16 进程下：

```

102 521.899805 528.519076 528.651497 539.016423
进程数: 16 | 矩阵规模: 2048 | 矩阵计算所消耗的时间: 117.624405 秒
user@user-virtual-machine:~/并行程序设计/hw2$

```

分析原因，推测可能是我的虚拟机核数限制导致的。由于我的虚拟机只有两个核，在多于两个进程时是不被允许运行，通过--oversubscribe 选项强制运行的。但是这会导致一些问题，由于 我只有两个核，如果运行多于两个进程，那么一个核上就会有多个进程，在同一个核上的进程 就不是真正意义上的并行，而是并发。这就导致进程之间的来回切换，运行时间最长的进程总是来自每个核上最晚运行完成的（进程较少时，如本实验的场景下，起始运行时刻基本一致）， 那这样相当于运行时间最长的进程走过了处在本核上所有进程运行的时间，因此没有任何性能 收益。并且频繁的进程切换开销反而导致运行性能下降。

我的虚拟机核数：

处理器

处理器数量(P):
2

每个处理器的内核数量(C):
1

处理器内核总数:
2

3. 实验结果

进程数	矩阵规模				
	128	256	512	1024	2048
1	0.010	0.085	0.83	7.78	189.75
2	0.005	0.063	0.55	5.02	110.00
4	0.008	0.076	0.61	5.25	112.94
8	0.061	0.110	0.68	5.60	113.15
16	0.116	0.146	0.73	6.33	117.62

分析上述实验结果，会发现 2 进程时相比于 1 进程性能有提升，但随着进程数增加，由于自身虚拟机核数限制，导致性能不再提升，同时由于进程切换开销和点对点通信开销，性能略有下降，尤其是矩阵规模不大时，由于计算开销很小，通信开销和进程切换开销变成主要影响因素，对整体程序运行的影响变大。

2. 内存有限情况下的大规模矩阵乘法方案以及大规模稀疏矩阵乘法性能提升方法的讨论

在内存有限时，分块矩阵乘法是一种有效的策略。其核心思想是将大矩阵分割成较小的子矩阵（块），每次只将部分块加载到内存中进行乘法运算。

当大规模稀疏矩阵计算场景下，可以换一种存储思路，即只存储非零数，并同时存储其索引，这样可以在内存有限的情况下进行矩阵乘法计算。

同时这种存储方式应该也可以用来提升大规模稀疏矩阵乘法的性能，在有序的存储情况下，索引查找采用二分只需要 $\log n$ 的复杂度，同时计算的数据量大幅减小，相比与索引开销的增加，计算量降低了更多，因此可以提升性能。

4. 实验感想

通过这次实验，我初步了解了 MPI 的使用方法，并行计算带来的性能提升，以及可能带来的附加通信开销对性能的影响。同时体会到并行的高效性要与硬件适配，不然效果并不会突出。同时探讨了内存有限情况下矩阵乘法方案以及大规模稀疏矩阵乘法性能提升方法和思路。受益匪浅。